

DOCUMENTO TÉCNICO · DESPLIEGUE

SJAP Balance

Despliegue, requisitos y ubicación del código

Guía para instalar SJAP Balance en otro equipo o llevarlo a producción en un ambiente distinto al actual. Estado verificado el 23 de septiembre de 2026 (versión 0.1.0).

ESTACIÓN	VERSIÓN	FECHA	PÚBLICO
EDS LA FLORIDA · Cota	0.1.0	28 de septiembre de 2026	Equipo técnico / TI

01 Dónde está el código y los recursos

PIEZA	UBICACIÓN	NOTAS
Código fuente	<code>/Users/ivanclabe/Workspace/sjap-balance</code> (Mac del desarrollador, usuario <code>ivanclabe</code>)	No es un repositorio git y no tiene copia remota (GitHub/GitLab). Hoy es la única copia: ver sección 9.
Base de datos	Supabase, proyecto <code>iosxchnwfvimfgozumqh</code> https://iosxchnwfvimfgozumqh.supabase.co	Proyecto compartido con otras apps (tablas <code>dk_*</code> , <code>ipler_*</code> , <code>sc_*</code> , <code>in_*</code> , logística). SJAP usa solo lo que empieza por <code>sjap_</code> .
Esquema de la base	<code>supabase/migrations/</code> (22 archivos)	Migraciones del proyecto (idénticas a las aplicadas) + 2 semillas para entornos nuevos. <code>supabase/instalacion/</code> trae la versión para pegar en el SQL Editor (<code>npm run generar:instalacion</code>).
Edge Functions	<code>supabase/functions/sjap-usuarios</code> , <code>supabase/functions/chat-asistente</code>	Desplegadas en el proyecto. <code>admin-crear-usuario</code> quedó retirada (responde 410) y se puede borrar desde el Dashboard.
Script de arranque	<code>supabase/scripts/crear_usuario_master.sql</code>	Crea el perfil del primer usuario master.
Variables de entorno	<code>.env</code> (no versionar) · plantilla en <code>.env.example</code>	<code>VITE_SUPABASE_URL</code> , <code>VITE_SUPABASE_ANON_KEY</code> .
Archivos de prueba	<code>test-fixtures/</code>	Cierre diario real (12-feb-2026) y balance mensual de ejemplo.
Frontend en producción	—	No hay despliegue publicado. Hoy corre en local con <code>npm run dev</code> .

Documentos complementarios (privados; se comparten desde el menú *Share*):

- Documento técnico completo: <https://claude.ai/artifact/Cjwn6VrdAy9mzZxF8Aizz3>
- Manual de usuario: <https://claude.ai/artifact/EyxwS4KwoUqPnxEYDs8rHH>

Estructura del proyecto

```

sjap-balance/
├── index.html           # fuentes IBM Plex + montaje de React
├── package.json         # scripts: dev, build, preview, test:parser
├── vite.config.js
├── .env.example
├── README.md
├── DESPLIEGUE.md       # este archivo
├── src/                # frontend (páginas, parser del .xlsx del POS, hooks)
├── supabase/
│   ├── migrations/     # esquema completo de la base
│   ├── functions/      # Edge Functions (Deno)
│   └── scripts/         # usuario master
└── test-fixtures/

```

02 Arquitectura

- **Frontend:** SPA estática (React 18 + Vite 5). Se compila a `dist/` y la sirve cualquier hosting. El archivo del POS se procesa en el navegador.
- **Backend:** Supabase. Postgres guarda los datos (32 tablas `sjap_*`, con RLS por estación y por rol). Auth maneja el ingreso: el usuario `master` entra como `master@sjap.local`. Hay dos Edge Functions: `sjap-usuarios` (gestión de cuentas) y `chat-asistente`.
- **Anthropic API:** solo la usa el asistente de chat (`chat-asistente`). Es opcional.

La `anon key` es pública (va dentro del bundle). Lo que protege los datos es la RLS. La `service_role key` solo existe dentro de las Edge Functions.

Autenticación

- **Identidad:** Supabase Auth (`auth.users`) es la fuente de verdad del correo y la contraseña. `sjap_usuarios` es solo el perfil de la app (usuario, nombre, rol, estado); no guarda contraseñas.
- **Ingreso:** con el correo registrado o, si la persona no tiene correo, con su nombre de usuario (correo interno `<usuario>@sjap.local`). Tener cuenta en Auth no basta: se exige perfil **activo**.
- **Recuperación:** "¿Olvidaste tu contraseña?" envía un enlace de Supabase al correo; la contraseña nueva se valida en el servidor. Requiere configurar en Supabase → Authentication → URL Configuration el *Site URL* y la URL de la app en *Redirect URLs*, y un SMTP propio (el correo integrado de Supabase solo entrega a miembros del equipo del proyecto y con un límite muy bajo). Quien no tiene correo pide a un master una contraseña temporal.
- **Contraseñas:** mínimo 8 caracteres, letras y números, no triviales. La regla la aplica la función `sjap-usuarios` en el servidor.
- **Contraseñas temporales:** los usuarios nuevos y las contraseñas restablecidas por un master quedan marcados; la app obliga a cambiarlas al entrar.
- **Desactivar** un usuario lo bloquea en Auth (no puede entrar ni renovar su sesión) y las políticas RLS dejan de devolverle datos de inmediato. Se conserva su historial.
- Una estación **siempre conserva al menos un master activo** (trigger en la base).
- La sesión se **cierra sola** tras 30 minutos sin actividad (configurable), con aviso 60 s antes.
- Toda acción sobre usuarios queda en `sjap_auditoria` (entidad `usuario`), sin contraseñas.

Roles y permisos (aplicados en la base de datos con RLS, no solo en pantalla)

ÁREA	MASTER	DEPENDIENTE
Consultar todos los módulos y reportes	Sí	Sí
Cargar archivos de cierre (también re-subir)	Sí	Sí
Lectura de tanque, efectivo, facturas y ventas efectivo/QR	Sí	Sí (crear)
Corregir o eliminar facturas/movimientos manuales	Sí	No
Corregir venta, galones o clientes de un cierre	Sí	No
Turnos y ausencias de hoy en adelante	Sí	Sí
Turnos y ausencias ya pasados	Sí	No (historial)
Promotores, islas, esquemas de turno, clientes, productos, medios de pago, presupuesto, configuración	Sí	Solo lectura
Usuarios	Sí	Solo su contraseña
Borrar historia (cierres, transacciones, auditoría)	Nunca por la app	No

Además: los datos importados del Excel no se editan ni se borran desde la app; los catálogos con historial solo se desactivan (renombrar una isla o un medio de pago que aparece en ventas históricas está bloqueado); y re-subir el archivo de un día **cerrado** solo actualiza su detalle, sin tocar los totales ni el balance conciliados.

03 Requisitos

Para desarrollar

- Node.js 18 o superior (recomendado 20 o 22 LTS). Probado con Node 26.7.0 y npm 11.19.0.
- Supabase CLI 2.x (probado con 2.114.0): `brew install supabase/tap/supabase`.
- Git.
- Docker, solo si quieres levantar la base de datos localmente.

Para producción

- Un proyecto Supabase dedicado. Se recomienda el plan Pro por los backups diarios.
- Un hosting estático con HTTPS y reescritura SPA (todas las rutas → `index.html`).
- Opcional: API key de Anthropic para el asistente.

Para los usuarios

- Navegador actualizado (Chrome, Edge, Firefox o Safari).
- Salida HTTPS hacia el dominio del hosting y hacia `<ref>.supabase.co`.
- Opcional: `fonts.googleapis.com` y `fonts.gstatic.com`.

Versiones principales: React 18.3.1 · Vite 5.4.21 · react-router-dom 6.30.6 · @supabase/supabase-js 2.112.4 · xlsx 0.18.5 · recharts 3.10.1.

04 Instalación local

A. Frontend local contra un Supabase en la nube

```
cp .env.example .env      # completar URL y anon key (Dashboard → Project Settings → API)
npm ci
npm run dev                # http://localhost:5173
npm run test:parser        # prueba el parser con test-fixtures/
```

B. Todo local (Docker)

```
supabase init              # solo si no existe supabase/config.toml
supabase start
supabase db reset          # aplica supabase/migrations/ desde cero
supabase status            # URL y anon key locales → úsalas en .env
```

Después crea el usuario master:

1. En Studio (<http://localhost:54323>), ve a Authentication → Add user y crea `master@sjap.local` con la opción *Auto Confirm User*.
2. Ejecuta `supabase/scripts/crear_usuario_master.sql`.

Probado el 23-sep-2026: las 21 migraciones se aplicaron sin errores sobre una base vacía. Resultado: 32 tablas, 124 políticas RLS y 19 funciones, igual que el proyecto actual, más las semillas (estación, 3 productos, 17 medios de pago, turnos T1–T4 y esquema «Estándar»).

05 Despliegue en un ambiente nuevo (producción)

1. **Crear el proyecto Supabase.** Región sugerida: `sa-east-1` (São Paulo) o `us-east-1`. Guarda la contraseña de la base en un gestor de secretos.
2. **Configurar Auth.** Deja habilitado el proveedor Email y **desactiva «Allow new users to sign up»**. Los usuarios los crea el master desde la app.
3. **Aplicar el esquema:**

```
supabase login
supabase init                # si falta supabase/config.toml
supabase link --project-ref <nuevo-ref>
supabase db push              # aplica las 22 migraciones
```

4. **Crear el usuario master.** En el Dashboard, ve a Authentication → Users → Add user, crea `master@sjap.local` con *Auto Confirm User* y luego ejecuta `supabase/scripts/crear_usuario_master.sql` en el SQL Editor.
5. **Desplegar las Edge Functions:**

```
supabase functions deploy sjap-usuarios
supabase functions deploy chat-asistente      # opcional
supabase secrets set ANTHROPIC_API_KEY=sk-ant-...  # solo si se usa el asistente
```

`SUPABASE_URL`, `SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY` las inyecta Supabase automáticamente.

6. Compilar el frontend con las claves del nuevo proyecto. Vite las incrusta al compilar, así que si cambian hay que volver a compilar.

```
VITE_SUPABASE_URL=https://<nuevo-ref>.supabase.co \
VITE_SUPABASE_ANON_KEY=<anon-key> \
npm ci && npm run build          # genera dist/
```

7. Publicar `dist/` en el hosting (sección 6) y hacer la verificación (sección 8).

Proyecto actual: las dos semillas nuevas (20260901153900, 20260917212955) tienen fechas anteriores a la última migración remota. Por eso `supabase db push` pedirá `--include-all`. Son idempotentes y no cambian nada donde los datos ya existen.

Otra estación: las semillas y la migración 20260910174944 usan el id fijo de EDS LA FLORIDA (884b3769-2c20-4a0b-8019-6972bf5e4caa). Para otra estación, cambia nombre y ciudad en la semilla y conserva el id.

06 Hosting del frontend (fallback SPA obligatorio)

La app usa rutas del navegador (`/diarios/2026-02-12`), así que toda ruta desconocida debe devolver `index.html`.

Vercel — `vercel.json`:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Netlify / Cloudflare Pages — `public/_redirects`:

```
/*    /index.html    200
```

Nginx:

```
server {
    listen 443 ssl http2;
    server_name sjap.midominio.com;
    root /var/www/sjap-balance/dist;
    location /assets/ { add_header Cache-Control "public, max-age=31536000, immutable"; }
    location / { try_files $uri $uri/ /index.html; add_header Cache-Control "no-cache"; }
}
```

IIS — `dist/web.config` (requiere el módulo URL Rewrite):

```
<configuration><system.webServer><rewrite><rules>
  <rule name="SPA" stopProcessing="true">
    <match url=".*" />
    <conditions logicalGrouping="MatchAll">
      <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
      <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
    </conditions>
    <action type="Rewrite" url="/index.html" />
  </rule>
</rules></rewrite></system.webServer></configuration>
```

07 Variables y secretos

NOMBRE	DÓNDE	VISIBILIDAD
VITE_SUPABASE_URL	Build del frontend	Pública
VITE_SUPABASE_ANON_KEY	Build del frontend	Pública (la protección real es la RLS)
ANTHROPIC_API_KEY	Secreto de Edge Functions	Secreta
SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY	Edge Functions (automáticas)	Secreta la service role; solo la usa sjap-usuarios, nunca el frontend
Contraseña de la base	Gestor de secretos	Secreta

`.gitignore` ya excluye `.env`, `.env.local`, `node_modules` y `dist`.

Los umbrales de alerta y los minutos de inactividad antes de cerrar la sesión (`sesion_inactividad_minutos`, 30 por defecto; 0 = nunca) no son variables de entorno. Viven en la tabla `sjap_estacion_config` (columnas `clave`, `valor`) y se cambian por SQL.

08 Verificación después de desplegar

- El usuario `master` puede entrar. Un usuario inexistente ve «Usuario o contraseña incorrectos.».
- Recargar con F5 en `/mensual` o `/diarios/2026-02-12` carga la app, no un 404 (fallback SPA).
- Sin sesión, la API no devuelve datos: `curl "$URL/rest/v1/sjap_cierre_diario?select=fecha&limit=1" -H "apikey: $ANON" -H "Authorization: Bearer $ANON" → []`
- Subir `test-fixtures/cierre-diario-2026-02-12.xlsx` en *Cargar* termina en «completado». Bórralo después si no corresponde.
- Crear un usuario dependiente de prueba: al entrar debe pedirle cambiar la contraseña temporal, y luego ver Configuración en solo lectura. Desactivarlo y confirmar que ya no puede entrar.
- Si se configuró la API key, el asistente responde.
- La consola del navegador y los logs de Edge Functions no muestran errores.

Publicar una versión nueva: `npm ci && npm run build`, luego reemplazar `dist/` en el hosting. Si hubo cambios en la base, correr `supabase db push`. Si cambió una función, correr `supabase functions deploy <nombre>`. Todo cambio de base debe ser un archivo nuevo en `supabase/migrations/` (`supabase migration new <nombre>`), nunca SQL suelto.

09 Pendientes antes de producción

PRIORIDAD	HALLAZGO	ACCIÓN
Crítica	El código no tiene control de versiones y existe una sola copia.	<code>git init</code> y push a un repositorio privado de la empresa.
Alta	El proyecto Supabase es compartido con otras apps. Ahí hay una tabla ajena a SJAP (<code>app_estado_workflow</code>) sin RLS.	Usar un proyecto dedicado para SJAP. Avisar al responsable de esa tabla.

PRIORIDAD	HALLAZGO	ACCIÓN
Alta	<code>xlsx</code> 0.18.5 de npm tiene vulnerabilidades conocidas (CVE-2023-30533, CVE-2024-22363).	Actualizar a SheetJS 0.20.x (<code>cdn.sheetjs.com</code>) y correr <code>npm run test:parser</code> .
Media	PostgreSQL devuelve máximo 1.000 filas por consulta. <code>sjap_transacciones</code> ya tiene 1.344.	Paginar las consultas de rangos amplios.
Media	Ajustes de Auth que solo se cambian en el Dashboard: el registro público está abierto (lo comparten otras apps del proyecto), la longitud mínima de contraseña de Supabase es 6 y la protección contra contraseñas filtradas está apagada.	En un proyecto dedicado: desactivar «Allow new users to sign up», subir el mínimo a 8 y activar <i>Leaked password protection</i> (plan Pro). La app ya aplica su propia política de 8 caracteres.
Baja	El SDK de Anthropic no tiene versión fijada en la función. El bundle pesa 1,4 MB.	Fijar la versión. Aplicar code-splitting si hace falta.

Para migrar el historial actual (28 cierres, 1.344 transacciones, etc.) a un proyecto nuevo, ver la sección «Migrar los datos actuales» del documento técnico completo. Ese procedimiento está documentado pero todavía no se ha ensayado.